

```
# Paramdeep Choudhary
# Cu24250162
```

```
# =====
# EXPERIMENT NO. 1
# IMPLEMENTATION OF FUNDAMENTAL IMAGE PROCESSING OPERATIONS
# USING PYTHON AND OPENCV
# =====

# -----
# 1. IMPORT REQUIRED LIBRARIES
# -----

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

from google.colab import files
from IPython.display import display

print("Libraries imported successfully!")

# -----
# 2. UPLOAD / ACQUIRE IMAGE
# -----

print("\nPlease upload a JPG or PNG image:")

uploaded = files.upload()

# Get uploaded filename
filename = list(uploaded.keys())[0]

print("\nImage uploaded successfully:", filename)

# -----
# 3. READ IMAGE USING OPENCV
# -----

image = cv2.imread(filename)

if image is None:
    raise Exception("Error: Image could not be loaded.")

print("Image loaded successfully!")

# -----
# 4. DISPLAY IMAGE USING OPENCV
```

```
# -----  
  
# OpenCV reads images in BGR format.  
# Convert BGR to RGB before displaying with Matplotlib.  
  
image_rgb = cv2.cvtColor(  
    image,  
    cv2.COLOR_BGR2RGB  
)  
  
plt.figure(figsize=(7, 5))  
plt.imshow(image_rgb)  
plt.title("Original Image")  
plt.axis("off")  
plt.show()  
  
# -----  
# 5. EXAMINE IMAGE PROPERTIES  
# -----  
  
height, width, channels = image.shape  
  
print("\n===== IMAGE PROPERTIES =====")  
  
print("Filename      :", filename)  
print("Height        :", height, "pixels")  
print("Width          :", width, "pixels")  
print("Resolution     :", width, "x", height)  
print("Number of      :", channels)  
print("channels")  
print("Data type      :", image.dtype)  
print("Total pixels   :", height * width)  
  
print("=====")  
  
# -----  
# 6. SAVE IMAGE IN JPEG AND PNG FORMAT  
# -----  
  
cv2.imwrite(  
    "output_image.jpg",  
    image  
)  
  
cv2.imwrite(  
    "output_image.png",  
    image  
)  
  
print("\nImage saved successfully!")  
print("JPEG file: output_image.jpg")  
print("PNG file : output_image.png")
```

```
# Check file sizes

jpeg_size = os.path.getsize(
    "output_image.jpg"
)

png_size = os.path.getsize(
    "output_image.png"
)

print("\nFile Size Comparison:")
print("JPEG size:", jpeg_size, "bytes")
print("PNG size :", png_size, "bytes")


# -----
# 7. CONVERT IMAGE TO GRAYSCALE
# -----

gray = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2GRAY
)

print("\nGrayscale conversion completed.")

print(
    "Grayscale image shape:",
    gray.shape
)


# -----
# 8. CONVERT IMAGE TO HSV
# -----

hsv = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2HSV
)

print("HSV conversion completed.")


# -----
# 9. CONVERT IMAGE TO LAB
# -----

lab = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2LAB
)

print("LAB conversion completed.")
```

```
# -----  
# 10. RESIZE IMAGE  
# -----  
  
# Resize to 50% of original dimensions  
  
new_width = width // 2  
new_height = height // 2  
  
resized = cv2.resize(  
    image,  
    (new_width, new_height)  
)  
  
print("\nResizing completed.")  
print(  
    "New resolution:",  
    new_width,  
    "x",  
    new_height  
)  
  
# -----  
# 11. ROTATE IMAGE  
# -----  
  
# Rotate image by 90 degrees clockwise  
  
rotated = cv2.rotate(  
    image,  
    cv2.ROTATE_90_CLOCKWISE  
)  
  
print("Rotation by 90 degrees completed.")  
  
# -----  
# 12. HORIZONTAL FLIP  
# -----  
  
horizontal_flip = cv2.flip(  
    image,  
    1  
)  
  
print("Horizontal flipping completed.")  
  
# -----  
# 13. VERTICAL FLIP  
# -----
```

```
vertical_flip = cv2.flip(
    image,
    0
)

print("Vertical flipping completed.")

# -----
# 14. IMAGE COMPLEMENT / NEGATIVE
# -----

negative = cv2.bitwise_not(
    image
)

print("Image negative/complement completed.")

# -----
# 15. CROP REGION OF INTEREST (ROI)
# -----

# We select the center portion of the image.
# This works automatically for different image sizes.

x1 = width // 4
y1 = height // 4

x2 = 3 * width // 4
y2 = 3 * height // 4

roi = image[
    y1:y2,
    x1:x2
]

print("ROI cropping completed.")

print(
    "ROI dimensions:",
    roi.shape[1],
    "x",
    roi.shape[0]
)

# -----
# 16. DISPLAY COLOR SPACE CONVERSIONS
# -----

plt.figure(figsize=(15, 10))

# Original
plt.subplot(2, 3, 1)
```

```
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

# Grayscale
plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

# HSV
# Convert HSV back to RGB only for visualization
hsv_rgb = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2RGB
)

plt.subplot(2, 3, 3)
plt.imshow(hsv_rgb)
plt.title("HSV")
plt.axis("off")

# LAB
# Convert LAB back to RGB for visualization
lab_rgb = cv2.cvtColor(
    lab,
    cv2.COLOR_LAB2RGB
)

plt.subplot(2, 3, 4)
plt.imshow(lab_rgb)
plt.title("LAB")
plt.axis("off")

# Negative
negative_rgb = cv2.cvtColor(
    negative,
    cv2.COLOR_BGR2RGB
)

plt.subplot(2, 3, 5)
plt.imshow(negative_rgb)
plt.title("Negative / Complement")
plt.axis("off")

# ROI
roi_rgb = cv2.cvtColor(
    roi,
    cv2.COLOR_BGR2RGB
)

plt.subplot(2, 3, 6)
plt.imshow(roi_rgb)
plt.title("Region of Interest (ROI)")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()

# -----
# 17. DISPLAY GEOMETRIC TRANSFORMATIONS
# -----

plt.figure(figsize=(15, 10))

# Original
plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original")
plt.axis("off")

# Resized
resized_rgb = cv2.cvtColor(
    resized,
    cv2.COLOR_BGR2RGB
)

plt.subplot(2, 3, 2)
plt.imshow(resized_rgb)
plt.title("Resized - 50%")
plt.axis("off")

# Rotated
rotated_rgb = cv2.cvtColor(
    rotated,
    cv2.COLOR_BGR2RGB
)

plt.subplot(2, 3, 3)
plt.imshow(rotated_rgb)
plt.title("Rotated 90°")
plt.axis("off")

# Horizontal Flip
horizontal_rgb = cv2.cvtColor(
    horizontal_flip,
    cv2.COLOR_BGR2RGB
)

plt.subplot(2, 3, 4)
plt.imshow(horizontal_rgb)
plt.title("Horizontal Flip")
plt.axis("off")

# Vertical Flip
vertical_rgb = cv2.cvtColor(
    vertical_flip,
    cv2.COLOR_BGR2RGB
)
```

```
plt.subplot(2, 3, 5)
plt.imshow(vertical_rgb)
plt.title("Vertical Flip")
plt.axis("off")
```

```
# ROI
plt.subplot(2, 3, 6)
plt.imshow(roi_rgb)
plt.title("Cropped ROI")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

```
# -----
# 18. DISPLAY ALL IMPORTANT RESULTS
# -----
```

```
plt.figure(figsize=(16, 12))
```

```
plt.subplot(3, 3, 1)
plt.imshow(image_rgb)
plt.title("Original")
plt.axis("off")
```

```
plt.subplot(3, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")
```

```
plt.subplot(3, 3, 3)
plt.imshow(hsv_rgb)
plt.title("HSV")
plt.axis("off")
```

```
plt.subplot(3, 3, 4)
plt.imshow(lab_rgb)
plt.title("LAB")
plt.axis("off")
```

```
plt.subplot(3, 3, 5)
plt.imshow(resized_rgb)
plt.title("Resized")
plt.axis("off")
```

```
plt.subplot(3, 3, 6)
plt.imshow(rotated_rgb)
plt.title("Rotated")
plt.axis("off")
```

```
plt.subplot(3, 3, 7)
plt.imshow(horizontal_rgb)
plt.title("Horizontal Flip")
```

```
plt.axis("off")

plt.subplot(3, 3, 8)
plt.imshow(vertical_rgb)
plt.title("Vertical Flip")
plt.axis("off")

plt.subplot(3, 3, 9)
plt.imshow(negative_rgb)
plt.title("Negative")
plt.axis("off")

plt.tight_layout()
plt.show()

# -----
# 19. DISPLAY ROI SEPARATELY
# -----

plt.figure(figsize=(6, 5))
plt.imshow(roi_rgb)
plt.title("Region of Interest (ROI)")
plt.axis("off")
plt.show()

# -----
# 20. SAVE PROCESSED IMAGES
# -----

cv2.imwrite(
    "grayscale.jpg",
    gray
)

cv2.imwrite(
    "hsv.jpg",
    hsv
)

cv2.imwrite(
    "lab.jpg",
    lab
)

cv2.imwrite(
    "resized.jpg",
    resized
)

cv2.imwrite(
    "rotated.jpg",
    rotated
)
```

```

cv2.imwrite(
    "horizontal_flip.jpg",
    horizontal_flip
)

cv2.imwrite(
    "vertical_flip.jpg",
    vertical_flip
)

cv2.imwrite(
    "negative.jpg",
    negative
)

cv2.imwrite(
    "roi.jpg",
    roi
)

print("\nAll processed images have been saved.")

# -----
# 21. FINAL OBSERVATIONS
# -----

print("""
=====
                        OBSERVATIONS
=====

1. The input image was successfully acquired and loaded
   using OpenCV.

2. The image dimensions, resolution, number of channels,
   and data type were successfully examined.

3. The image was successfully stored in JPEG and PNG
   formats.

4. The image was converted from BGR to Grayscale, HSV,
   and LAB color spaces.

5. Grayscale conversion reduced the image to a single
   intensity channel.

6. The image was resized to 50% of its original size.

7. The image was rotated by 90 degrees.

8. Horizontal and vertical flipping were successfully
   performed.

```

9. The complement/negative image was generated by inverting the pixel intensity values.
10. A Region of Interest (ROI) was successfully cropped from the center of the image.
11. These preprocessing operations are useful for preparing images for feature extraction, object detection, segmentation, and deep learning.

```
=====
                        EXPERIMENT COMPLETED SUCCESSFULLY
=====
""")
```

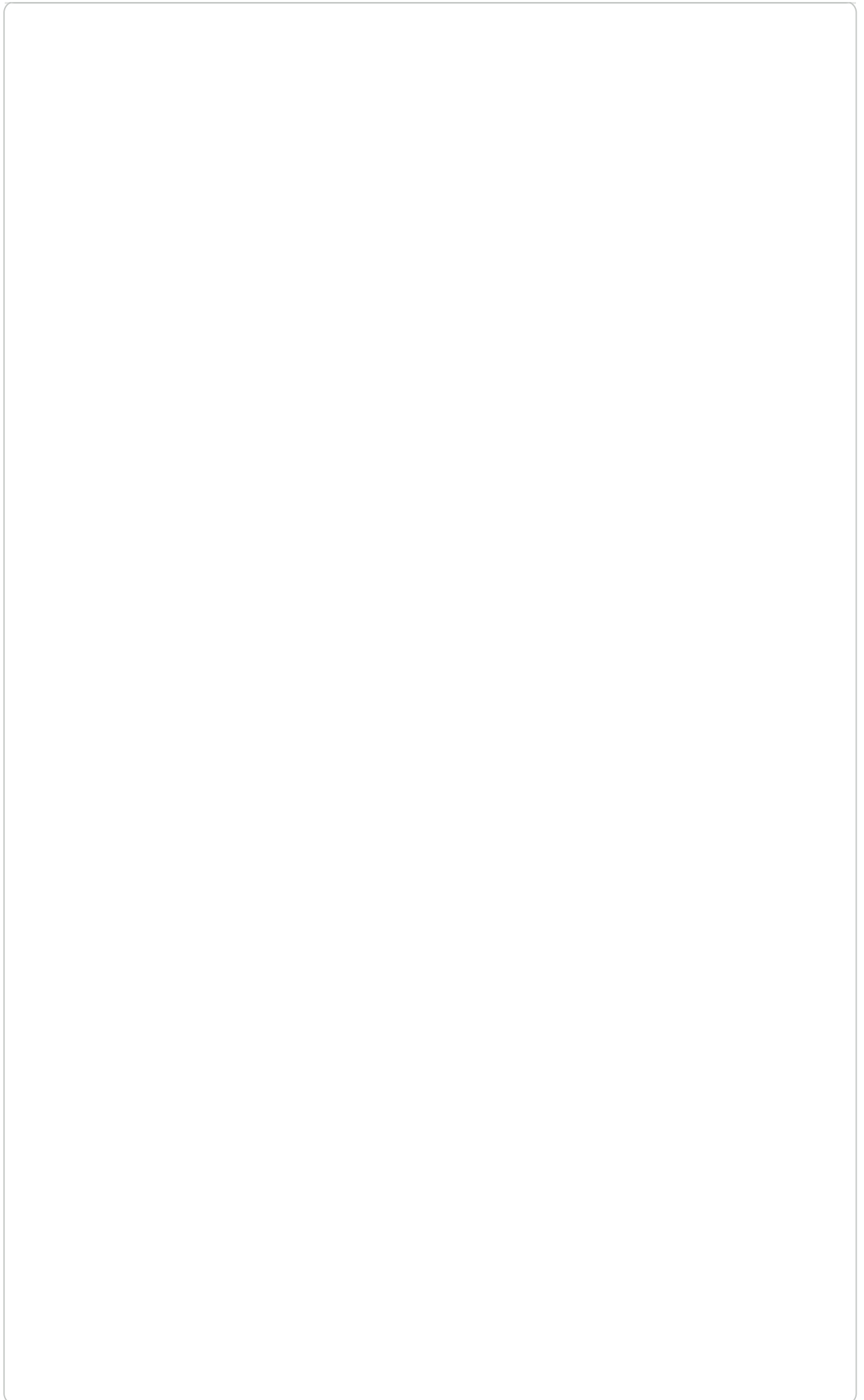
```
# -----
# 22. DOWNLOAD PROCESSED FILES (OPTIONAL)
# -----
```

```
print("\nYou can download individual processed images from")
print("the Colab file section if required.")
```

```
print("\nFiles created:")
```

```
files_created = [
    "output_image.jpg",
    "output_image.png",
    "grayscale.jpg",
    "hsv.jpg",
    "lab.jpg",
    "resized.jpg",
    "rotated.jpg",
    "horizontal_flip.jpg",
    "vertical_flip.jpg",
    "negative.jpg",
    "roi.jpg"
]
```

```
for f in files_created:
    print("-", f)
```



Libraries imported successfully!

Please upload a JPG or PNG image:

image.jpg

image.jpg(image/jpeg) - 398639 bytes, last modified: 10/09/2026 - 100% done
Saving image.jpg to image.jpg

Image uploaded successfully: image.jpg

Image loaded successfully!

Original Image



```
===== IMAGE PROPERTIES =====  
Filename      : image.jpg  
Height       : 853 pixels  
Width        : 1280 pixels  
Resolution   : 1280 x 853  
Number of    : 3  
channels  
Data type    : uint8  
Total pixels : 1091840  
=====
```

Image saved successfully!
JPEG file: output_image.jpg
PNG file : output_image.png

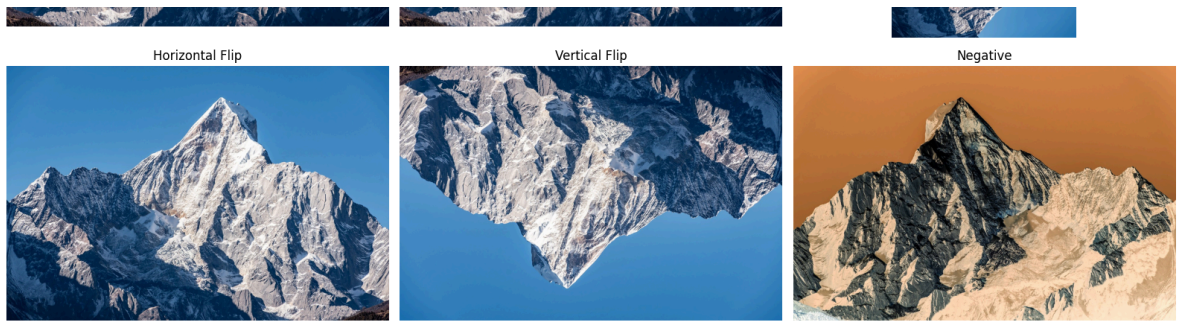
File Size Comparison:
JPEG size: 502084 bytes
PNG size : 1949030 bytes

Grayscale conversion completed.
Grayscale image shape: (853, 1280)
HSV conversion completed.
LAB conversion completed.

Resizing completed.
New resolution: 640 x 426
Rotation by 90 degrees completed

rotation by 90 degrees completed.
Horizontal flipping completed.
Vertical flipping completed.
Image negative/complement completed.
ROI cropping completed.
ROI dimensions: 640 x 426





Region of Interest (ROI)



All processed images have been saved.

OBSERVATIONS

1. The input image was successfully acquired and loaded using OpenCV.
2. The image dimensions, resolution, number of channels, and data type were successfully examined.
3. The image was successfully stored in JPEG and PNG formats.
4. The image was converted from BGR to Grayscale, HSV, and LAB color spaces.
5. Grayscale conversion reduced the image to a single intensity channel.
6. The image was resized to 50% of its original size.
7. The image was rotated by 90 degrees.
8. Horizontal and vertical flipping were successfully performed.
9. The complement/negative image was generated by inverting the pixel intensity values.

10. A Region of Interest (ROI) was successfully cropped from the center of the image.
11. These preprocessing operations are useful for preparing images for feature extraction, object detection, segmentation, and deep learning.

=====

EXPERIMENT COMPLETED SUCCESSFULLY

=====

You can download individual processed images from the Colab file section if required.

Files created:

- output_image.jpg
- output_image.png
- grayscale.jpg
- hsv.jpg
- lab.jpg
- resized.jpg
- rotated.jpg
- horizontal_flip.jpg
- vertical_flip.jpg
- negative.jpg
- roi.jpg

```
// 1. What is a digital image? Differentiate between grayscale and color images.
//
// Answer:
// A digital image is an image represented using pixels.
// Grayscale images have one channel, while color images usually have three channels.

// 2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.
//
// Answer:
// RGB = Red, Green, Blue.
// BGR = Blue, Green, Red (used by OpenCV).
// HSV = Hue, Saturation, Value.
// LAB = Lightness and two color components.

// 3. What is the purpose of converting an image to grayscale before processing?
//
// Answer:
// Grayscale reduces the image to one channel, making processing faster and simpler.

// 4. Explain image complement (negative image) and its applications.
//
// Answer:
// A negative image reverses pixel values.
// Formula: Negative = 255 - Original.
// It is used in medical imaging and image enhancement.

// 5. Differentiate between image resizing, cropping, and scaling.
//
// Answer:
// Resizing changes image dimensions.
// Cropping selects a specific part of an image.
// Scaling enlarges or reduces an image by a factor.

// 6. What is ROI, and why is it important?
//
// Answer:
// ROI (Region of Interest) is a selected part of an image for analysis.
// It focuses processing on important areas and saves time.

// 7. Why is OpenCV preferred for computer vision applications?
//
// Answer:
// OpenCV is free, fast, cross-platform, and provides many image and video processing functions.

// 8. How do image resolution and pixel intensity influence image quality?
//
// Answer:
// Higher resolution provides more detail.
// Pixel intensity controls brightness.
// In an 8-bit image, 0 is black and 255 is white.

// 9. Mention five real-world applications of image preprocessing.
//
// Answer:
```